Cloud Architecture

Assignment-2 (Report)

Submitted By: Fahamidul Hasan (20115491)

Contents

Application Scenario 2

Architecture Diagram 2

Workflow 3

AWS Well Architected Framework 8

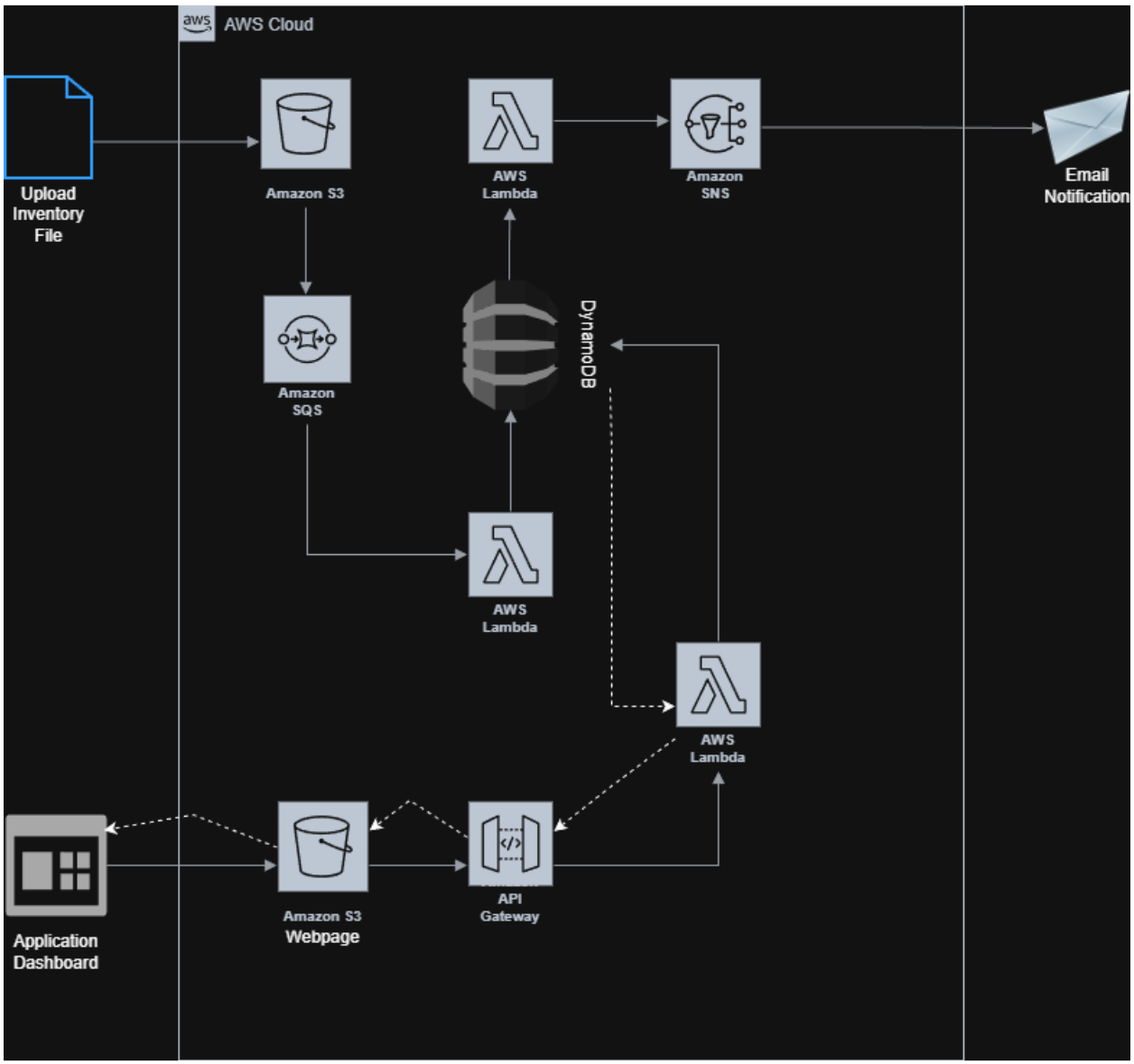
Conclusion 9

Plagiarism Declaration 9

Application Scenario

The scenario is based on a library inventory system for which a serverless AWS solution is proposed. The library may have multiple branches in different cities, in case any branch is out of stock for a specific book, an inventory alert will be sent to the designated email. The inventory details of all branches can also be viewed through a web page.

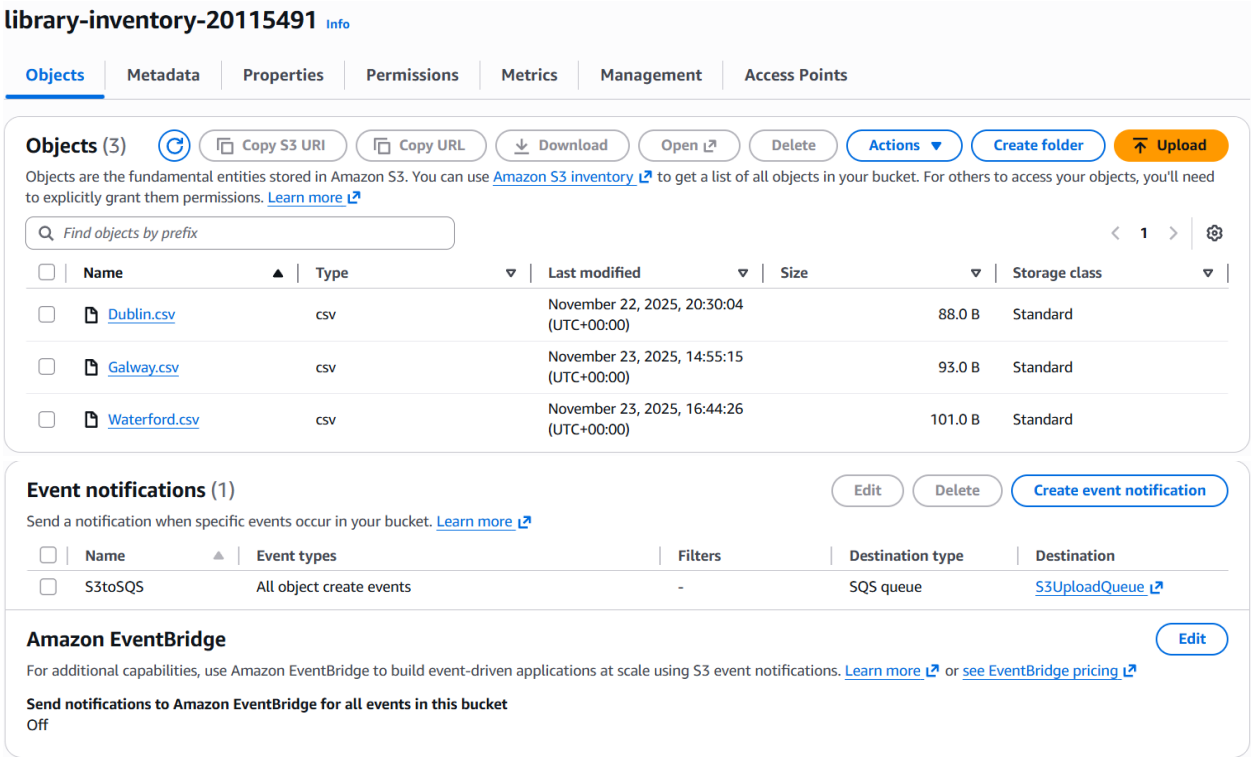
Architecture Diagram



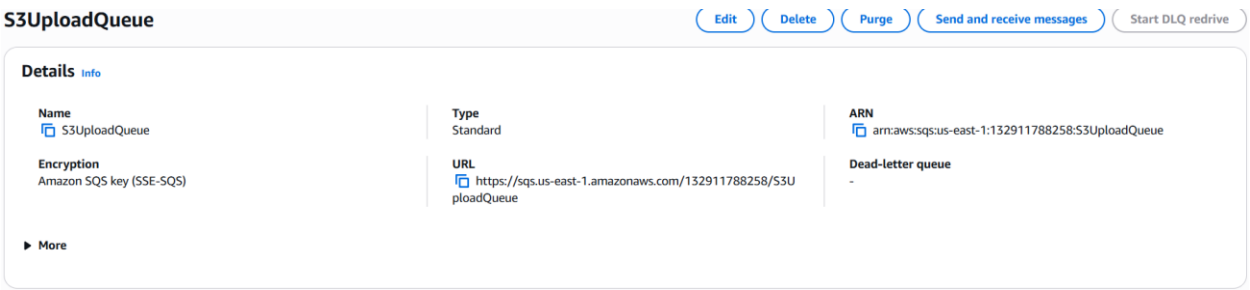
The architecture diagram lists all AWS services used for this serverless solution and visualizes their workflow. In this architecture I have used Amazon S3, Amazon SQS, AWS Lambda, DynamoDB, Amazon SNS, API Gateway to build a serverless loosely coupled system.

Workflow

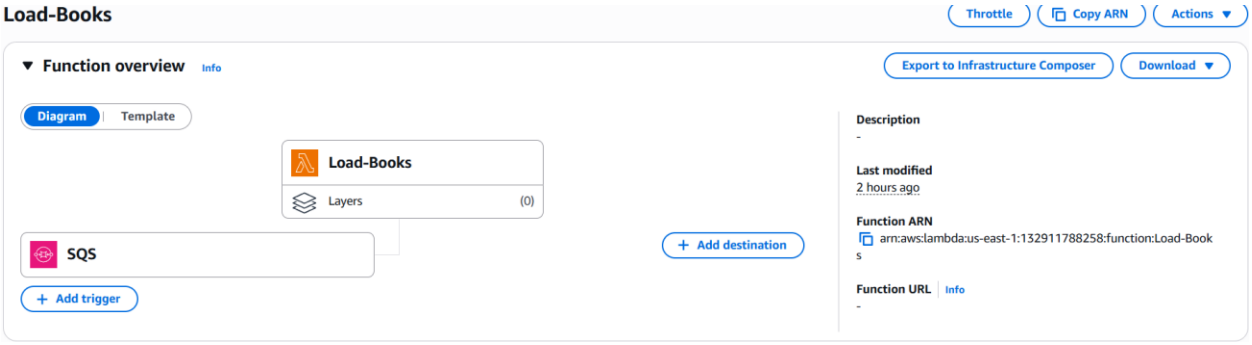
- ⇨ Users upload branch specific CSV files to Amazon S3 bucket. The S3 Bucket has an event notification set to S3toSQS queue on all objects create events.



- ⇨ SQS stores the messages in a Queue. This ensures that the architecture is loosely coupled as the producer (S3) and the consumer (Lambda) now have a buffer (SQS) between them.



⇨ Lambda is triggered by SQS. It picks up the batch of messages and processes them and finally uploads them to DynamoDB.




```
try:
    with open(local_filename) as csvfile:
        reader = csv.DictReader(csvfile, delimiter=',')
        for row in reader:
            print("CSV Row:", row)

            libraryTable.put_item(
                Item={
                    'Library': row['library'],
                    'Book': row['book'],
                    'Count': int(row['count'])
                }
            )
            total_records += 1
```

The code snippet shows the phase of writing new data to DynamoDB table.
=> Open uploaded CSV file
=> Read each row
=> print the entire row to CloudWatch
=> Insert each row to DynamoDB table
=> Increment record counter that can later be printed to CloudWatch log

⇨ DynamoDB table storing the records

Table: LibraryBooks - Items returned (9)

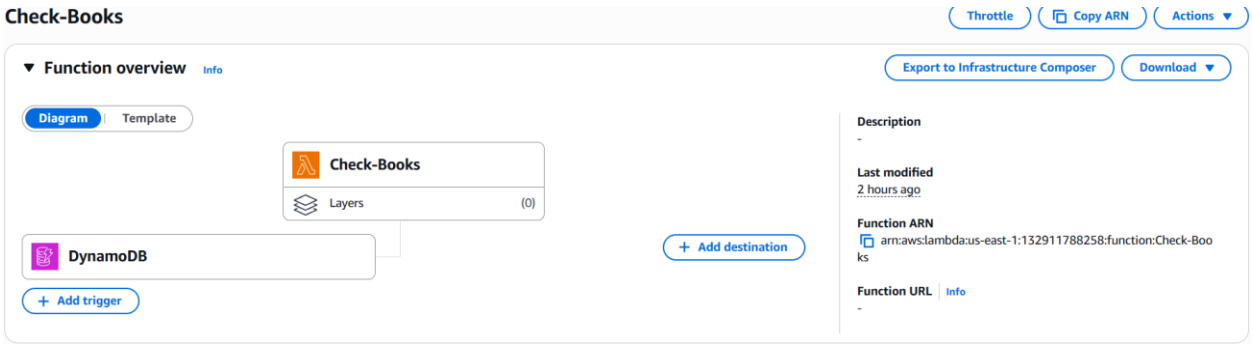
Actions ▼ | Create item

Scan started on November 27, 2025, 14:53:51

1

Library *(String)*	Book *(String)*	Count
Dublin	Harry Potter	23
Dublin	The Hobbit	10
Dublin	The Witcher	0
Galway	A Song of Ice and Fire	0
Galway	Hobbit	5
Galway	The Witcher	10
Waterford	A Song of Ice and Fire	0
Waterford	Hobbit	5
Waterford	The Witcher	0

⇨ DynamoDB then triggers another Lambda function



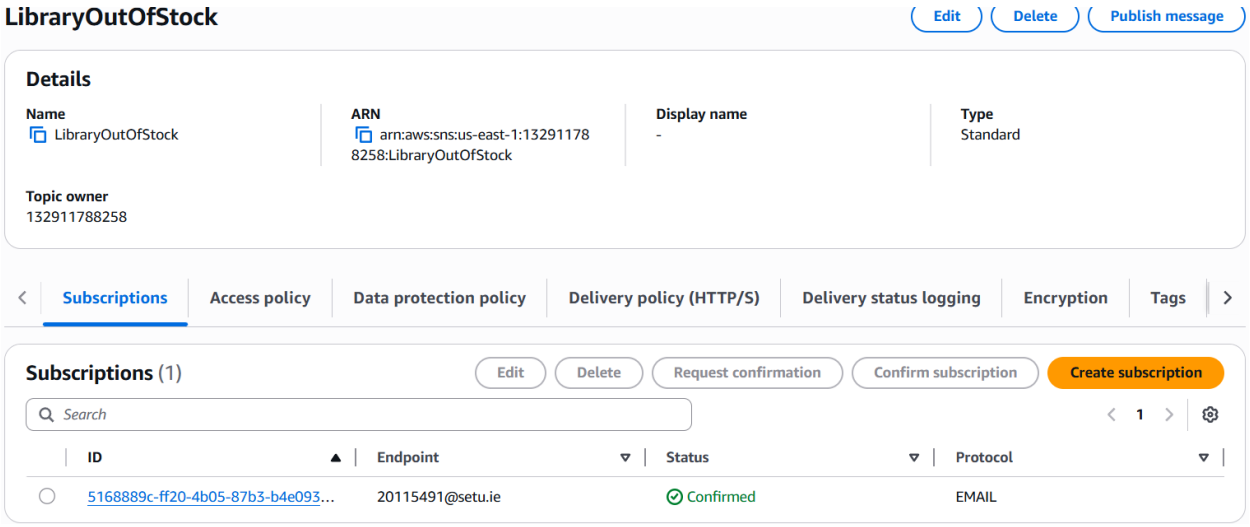

```python
for record in event['Records']:
    newImage = record['dynamodb'].get('NewImage', {})
    count = int(newImage['Count']['N'])

    if count == 0:
        library = newImage['Library']['S']
        book = newImage['Book']['S']

        message = f"OUT OF STOCK: '{book}' at {library} library."
        print(message)

        sns.publish(
            TopicArn=topic_arn,
            Message=message,
            Subject="Inventory Alert!"
        )
```

⇨ The code snippet shows the phase of publishing an SNS notification based on if any row contains count = 0

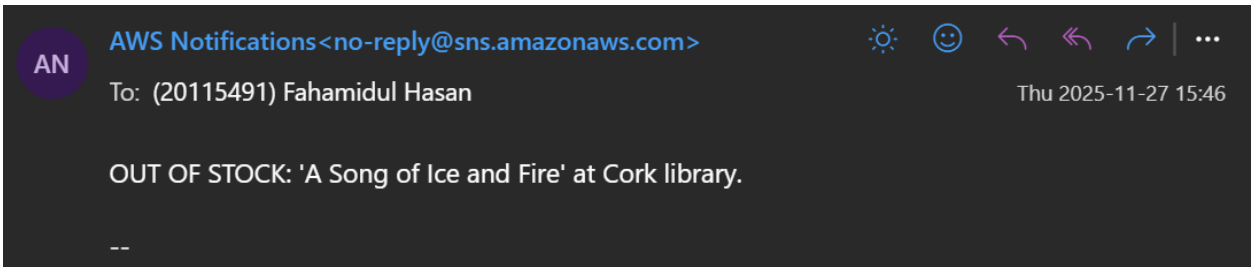
=> Read a new row

=> Convert value of count to int [Default is Number: N]

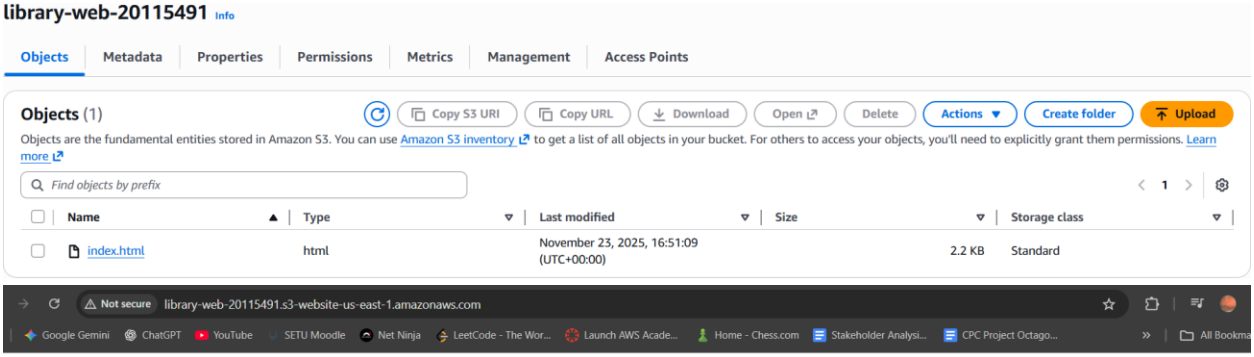
=> If count = 0 then publish that the book is out of stock at that library through SNS



An email is then sent through the SNS to the designated email address as Inventory alert.



⇨ To visualize the inventory, a static web page is hosted in an S3 Bucket.

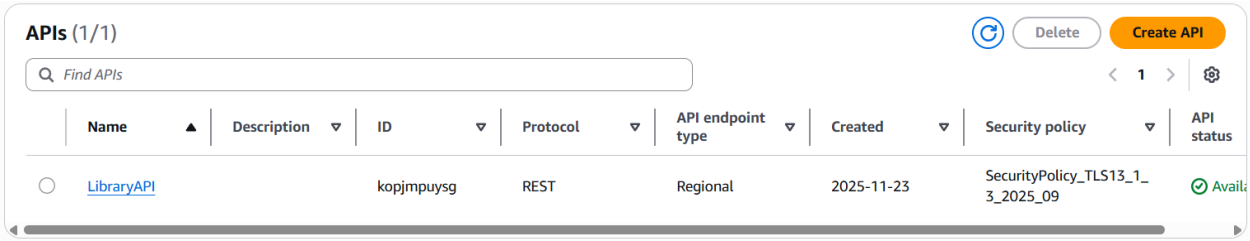


Library Inventory Viewer

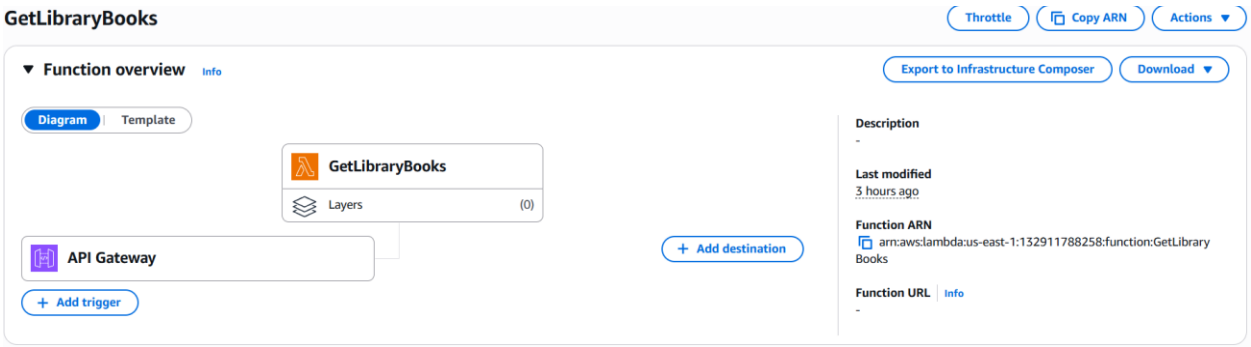
Select Library: All Libraries Load Books

Library	Book	Count
Cork	A Song of Ice and Fire	0
Cork	Hobbit	5
Cork	The Witcher	11
Dublin	Harry Potter	23
Dublin	The Hobbit	10
Dublin	The Witcher	0
Galway	A Song of Ice and Fire	0
Galway	Hobbit	5
Galway	The Witcher	10
Waterford	A Song of Ice and Fire	0
Waterford	Hobbit	5
Waterford	The Witcher	0

To retrieve the data from DynamoDB database, API Gateway is used.



The API Gateway triggers another Lambda function that fetches the data from DynamoDB table.



The code snippet displays the phase of scanning for items in the database and returning the data back to the front end.

=> Read query parameters

=> If library name is provided then query in that library for books

=> Else query for all books

=> Return the data back to front end as JSON

AWS Well Architected Framework

Evaluation of the system based on WAF:

- **Operational Excellence:** The system uses serverless components with CloudWatch logging to make it easy to monitor and troubleshoot.
- **Security:** Access to DynamoDB is controlled, which makes the system secure.
- **Reliability:** The SQS buffer ensures that no messages are lost in case lambda fails ensuring reliability.
- **Performance Efficiency:** Serverless components scale automatically according to performance needs.
- **Cost Optimization:** Serverless components are charged according to usage, optimizing cost.
- **Sustainability:** Using serverless resources on demand has minimal environmental impact.

Conclusion

In conclusion, I have designed a cloud solution that aligns with the well architected framework, and I have also used serverless components to build a loosely coupled system.

Plagiarism Declaration

I certify that this assignment is all my own work and contains no plagiarism. By submitting this assignment, I agree to the following terms: Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals and the internet) have been clearly acknowledged and referenced as such in the text by the use of 'quotation marks' (or indented italics for longer quotations) followed by the author's name and date [e.g. (Byrne, 2018)] either in the text or in a footnote/endnote. These details are then confirmed by a fuller reference in the bibliography. I have read Appendix 1 of the SETU Student Academic Misconduct Policy and Disciplinary Procedure, and I understand that only assignments which are free of plagiarism will be awarded marks. I further understand that the disciplinary procedure can lead to the suspension or permanent expulsion of students in serious cases.

- Fahamidul Hasan (20115491)